

STREAMLY

Demo Presentation II

Created by Maryam

Team Members: Devin, Jonathan, Ashwin, Avis



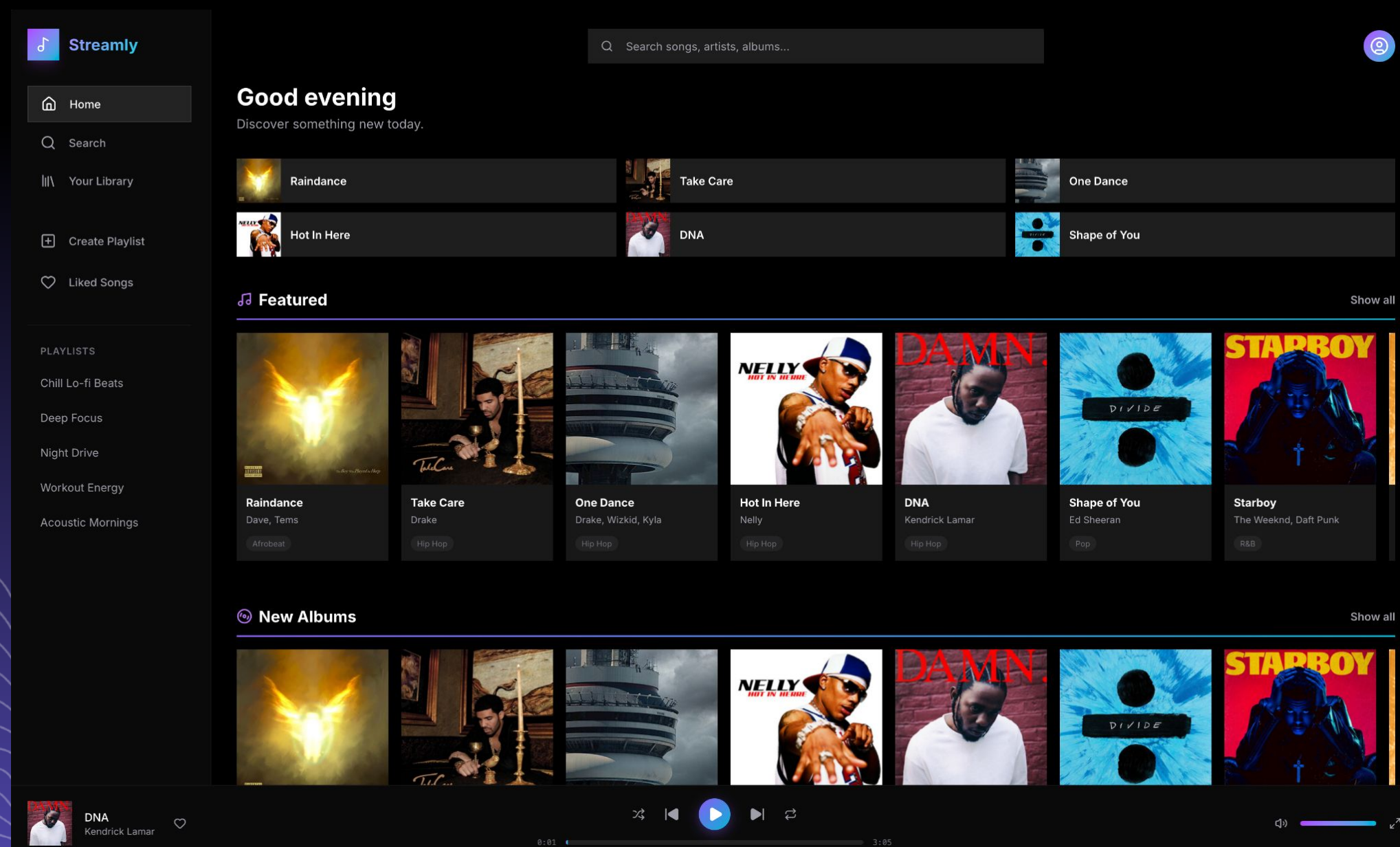


Part I: Customer Update & Progress



PRIMARY FOCUS

- IMPLEMENTING SEARCH & FILTERING FUNCTIONALITY
- IMPROVING USER EXPERIENCE & PERFORMANCE
- STRENGTHENING TESTING & DEPLOYMENT





ITERATION 1 SUMMARY (COMPLETED)

Feature	User Story
S1 Registration	As a new user, I want to create an account, so that I can login and access personalized features.
S2 Login	As a registered user, I want to log into my account securely, so that I can access my library and favourites.
S3 Logout	As a logged-in user, I want to be able to log out, so that my account remains secure.
S4 Admin Login	As an admin, I want to log in with my admin credentials aso that I can personally manage the system database.
S14 Adding Songs	As an admin, I want to add new songs to the database, so that all users can have access to real music.
S15 Editing Songs	As an admin, I want to edit song information (year, genre, artist) so that the database information stays accurate.
S16 Removing Songs	As an admin, I want to be able to remove songs, or song information from the system, so that outdated, controversial or incorrect content can be deleted.

ITERATION 2 SUMMARY



Feature	User Story	Status
S5 Search by song title	As a user I want to be able to search for a song by title, so that I can instantly find music I know.	DONE
S6 Search by artist name	As a user, I want to be able to search for a song by the artists' name, so that I can explore all songs by that artist.	DONE
S7 Search by album	As a user I want to be able to search songs by album so that I can find the song I'm looking for as well as the album its part of.	IN-PROGRESS
S8 Filter by genre	As a user, I want to filter songs by the genre, so that I can browse songs based on the mood.	DONE
S9 Flexible Search	As a user, I want to search for songs using different inputs like song title, artist name, lyrics, or genre.	DONE
S10 Filter by year	As a user, I want to filter songs by the release date, so that I can explore music from specific time periods.	DONE
S11 Filter by trend	As a user I want to filter songs by popularity so that I can know the trending songs.	DONE



CURRENT PROGRESS

BOARD AFTER ITERATION 2

Feature Planning ▾					Status board	Roadmap	Bugs 🐛	In review	My items	+ New view
🔍 Filter by keyword or by field										
Title	...	Status	⇅	...	Assignees	...				
▼ No Priority 18 Estimate: 0 ...										
1	🕒 S11 Trend Filtering (2 Hours) #4	Backlog	▼		AvisS12	▼				
2	🕒 S12 Favouritizing (2 Hours) #6	Backlog	▼		maryammarji	▼				
3	🕒 S13 Viewing Favourites (4 Hours) #8	Backlog	▼		maryammarji	▼				
4	🕒 S17 Play Music #14	Backlog	▼		Focusucof	▼				
5	🕒 S14 Searching Playlist (2 Hours) #13	In progress	▼		Johnathan-Lavoie	▼				
6	🕒 S7: Search Album #11	In progress	▼		Johnathan-Lavoie	▼				
7	🕒 S8: Flexible Search #16	In progress	▼		AvisS12	▼				
8	🕒 S5 Searching by Title #7	In Testing	▼		Johnathan-Lavoie	▼				
9	🕒 S6: Search Artist #9	In Testing	▼		Johnathan-Lavoie	▼				
10	🕒 S9: Genre Filtering #17	In Testing	▼		maryammarji	▼				
11	🕒 S10: Year Filtering #18	In Testing	▼		AvisS12	▼				
12	✅ S1 registration #1	Done	▼		Focusucof	▼				
13	✅ S2: Login #2	Done	▼		Focusucof	▼				
14	✅ S4 Admin Login #5	Done	▼		Ashwins1971 and F...	▼				
15	✅ S3 Logout #3	Done	▼		Focusucof	▼				
16	✅ S18 Password Encryption #15	Done	▼		Ashwins1971	▼				
17	✅ S15 Adding Songs #10	Done	▼		maryammarji	▼				
18	✅ S16 Editing Songs #12	Done	▼		Ashwins1971	▼				

CURRENT BOARD

Feature Planning ▾					Status board	Roadmap	Bugs 🐛	In review	My items	+ New view
🔍 Filter by keyword or by field										
Title	...	Status	⇅	...	Assignees	...				
▼ No Priority 18 Estimate: 0 ...										
1	🕒 S14 Searching Playlist (2 Hours) #13	Backlog	▼		Johnathan-Lavoie	▼				
2	🕒 S11 Trend Filtering (2 Hours) #4	Backlog	▼		AvisS12	▼				
3	🕒 S12 Favouritizing (2 Hours) #6	Backlog	▼		maryammarji	▼				
4	🕒 S13 Viewing Favourites (4 Hours) #8	Backlog	▼		maryammarji	▼				
5	🕒 S7: Search Album #11	In progress	▼		Johnathan-Lavoie	▼				
6	🕒 S17 Play Music #14	In progress	▼		Focusucof	▼				
7	✅ S10: Year Filtering #18	In Testing	▼		AvisS12	▼				
8	✅ S8: Flexible Search #16	Done	▼		AvisS12	▼				
9	✅ S5 Searching by Title #7	Done	▼		Johnathan-Lavoie	▼				
10	✅ S1 registration #1	Done	▼		Focusucof	▼				
11	✅ S16 Editing Songs #12	Done	▼		Ashwins1971	▼				
12	✅ S15 Adding Songs #10	Done	▼		maryammarji	▼				
13	✅ S2: Login #2	Done	▼		Focusucof	▼				
14	✅ S4 Admin Login #5	Done	▼		Ashwins1971 and F...	▼				
15	✅ S3 Logout #3	Done	▼		Focusucof	▼				
16	✅ S18 Password Encryption #15	Done	▼		Ashwins1971 and F...	▼				
17	✅ S6: Search Artist #9	Done	▼		Johnathan-Lavoie	▼				
18	✅ S9: Genre Filtering #17	Done	▼		maryammarji	▼				



Part II: Build & Deployment (Build Scripts)



Full Docker Compose File

```
services:
  >Run Service
  frontend:
    build: ./frontend
    ports:
      - "3000:3000"
    environment:
      - AUTH_INTERNAL_URL=http://auth:3001
    depends_on:
      - auth
      - backend
  >Run Service
  auth:
    build: ./auth
    ports:
      - "3001:3001"
    environment:
      - PORT=3001
    volumes:
      - ./data:/data
  >Run Service
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    volumes:
      - ./data:/data
```

How it works:

- Starts 3 services
- Maps the ports:
 - 3000 → frontend
 - 3001 → authentication
 - 8008 → backend
- Shares data using volumes



Frontend Docker Build

Node/Next.js

How it works:

- Uses a multi-stage build
- Install the dependencies
- Builds app (npm run build)
- Runs on an optimized production server
- Exposes port 3000

```
1  # ---- Dependencies ----
2  FROM node:20-alpine AS deps
3  WORKDIR /app
4  COPY package.json package-lock.json ./
5  RUN npm ci
6
7  # ---- Build ----
8  FROM node:20-alpine AS builder
9  WORKDIR /app
10 COPY --from=deps /app/node_modules ./node_modules
11 COPY . .
12 RUN npm run build
13
14 # ---- Production ----
15 FROM node:20-alpine AS runner
16 WORKDIR /app
17 ENV NODE_ENV=production
18
19 COPY --from=builder /app/.next/standalone ./
20 COPY --from=builder /app/.next/static ./next/static
21 COPY --from=builder /app/public ./public
22
23 EXPOSE 3000
24 CMD ["node", "server.js"]
```



Authorizing Backend

Bun/Typescript

How it works:

- Uses Bun runtime (faster than Node)
- Installs dependencies on lockfile
- Runs src/main.ts
- Exposes port 3001

```
1  FROM oven/bun:1 AS base
2  WORKDIR /app
3
4  COPY package.json bun.lock ./
5  RUN bun install --frozen-lockfile
6
7  COPY . .
8
9  EXPOSE 3001
10 ENV PORT=3001
11 CMD ["bun", "run", "src/main.ts"]
12
```



Backend Docker Build

Uses Java/Maven

```
1 # ---- Build ----
2 FROM maven:3.9-eclipse-temurin-21 AS build
3 WORKDIR /app
4 COPY pom.xml .
5 RUN mvn dependency:go-offline -B
6 COPY src ./src
7 RUN mvn package -DskipTests -B
8
9 # ---- Production ----
10 FROM eclipse-temurin:21-jre-alpine
11 WORKDIR /app
12 COPY --from=build /app/target/*.jar app.jar
13
14 EXPOSE 8080
15 CMD ["java", "-jar", "app.jar"]
16
```

How it works:

- Uses a Maven to build JAR
- Skips steps during docker build
- Runs with `java -jar app.jar`
- exposes port 8080



Part III: Devin's Application Demo

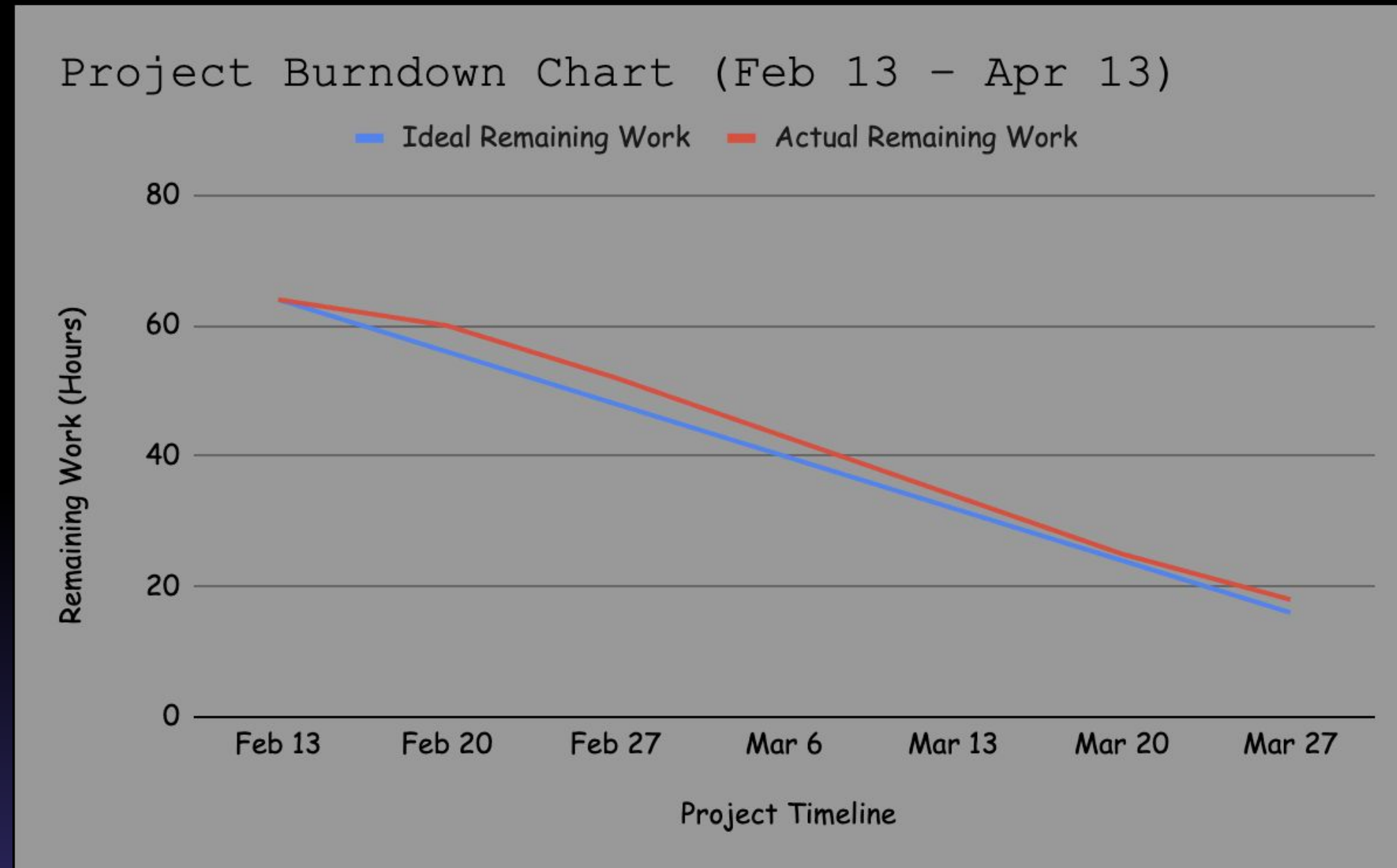
A quick demonstration of iteration 2 features
including search functionality, filtering & UI updates
for better browsing!



Part IV: Quick Diagram Demo + Burndown Chart



Burn-Down Chart



64 Hours = whole project estimate

24 Hours = Iteration 1

22 Hours = Iteration 2

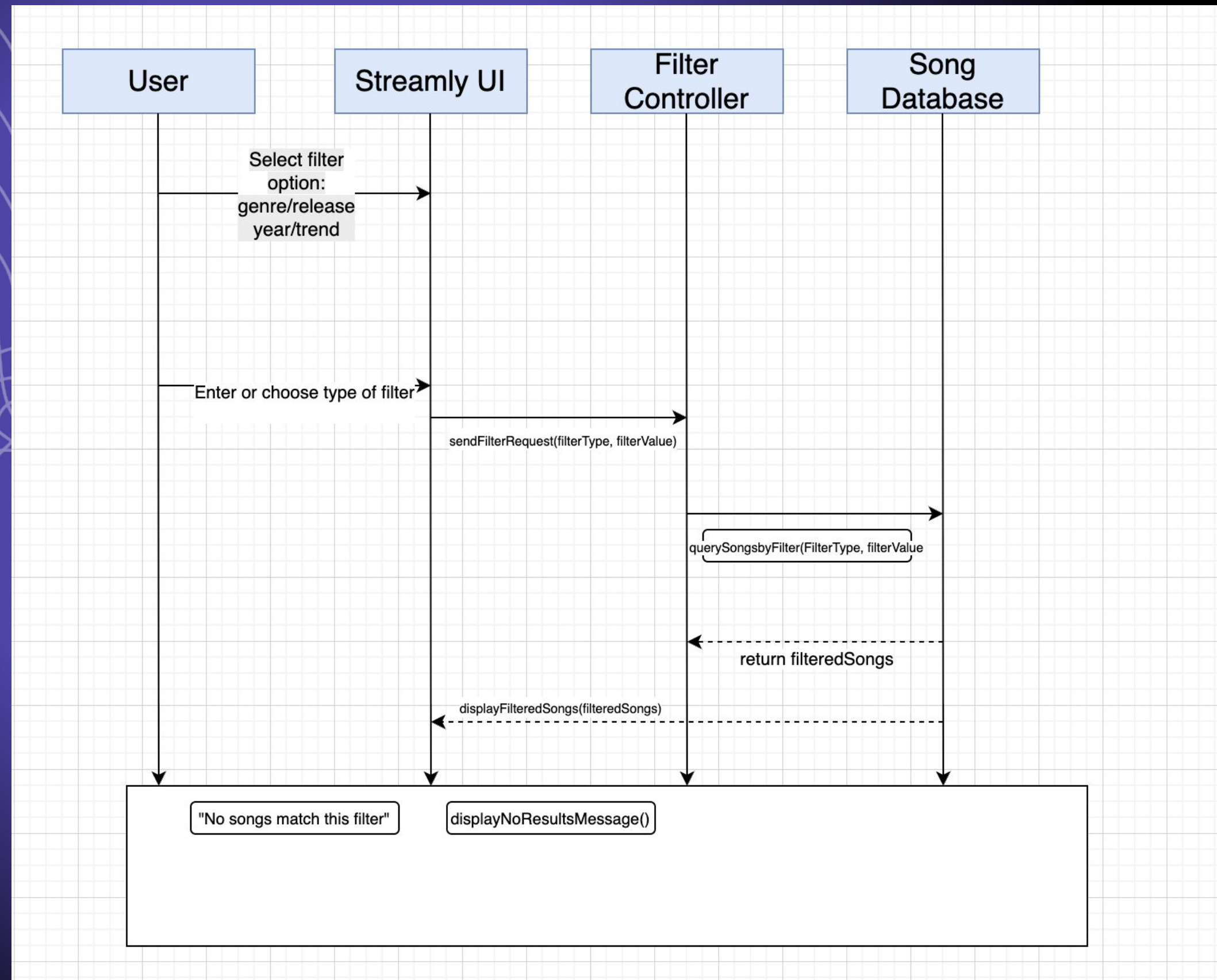
Average Velocity = $(24 + 22) / 2 = 23$ hours per iteration



UML Sequence for Filtering

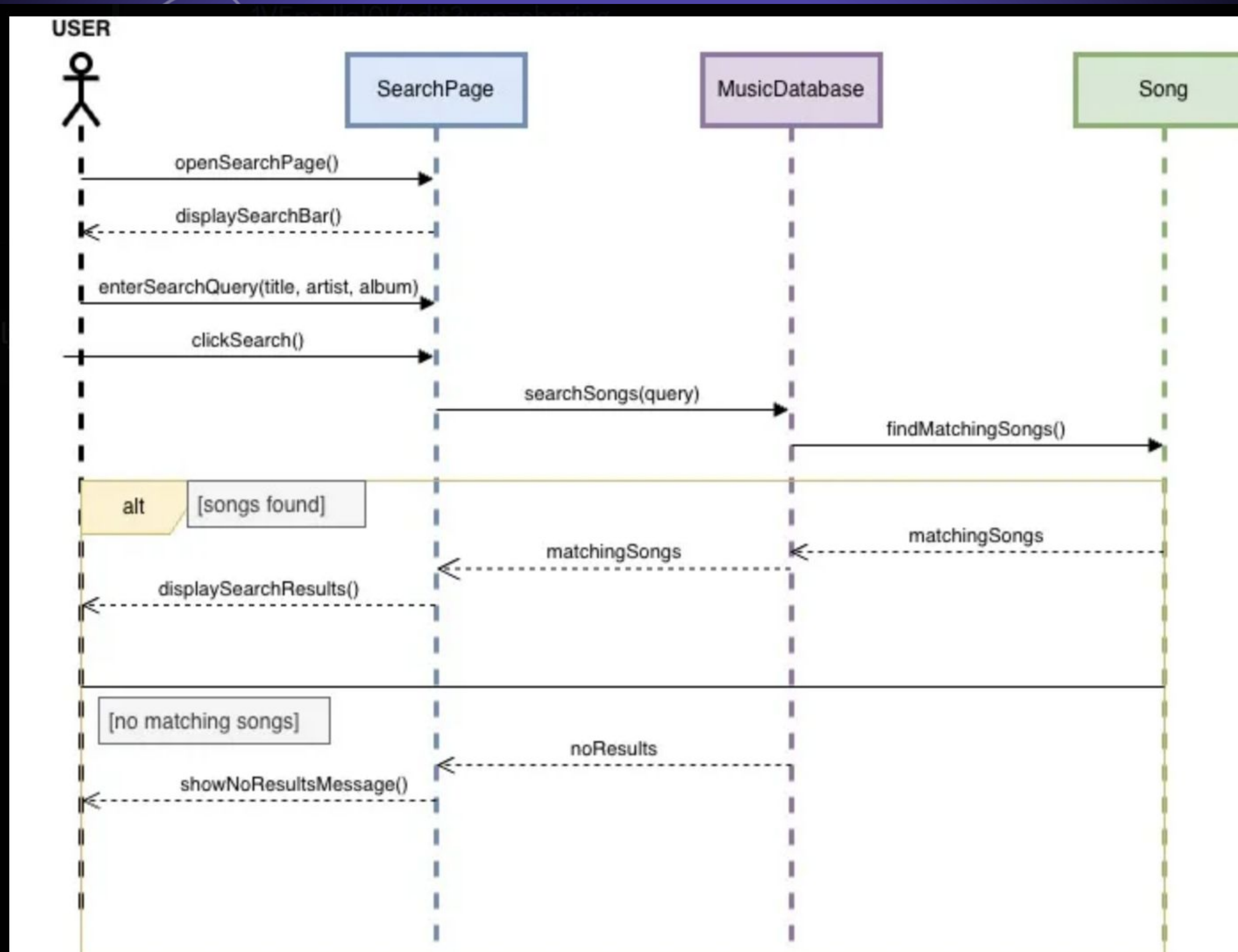
Coded Visual

```
if (songsFound) {
    displaySongs();
} else {
    displayNoResultsMessage();
}
```





UML Sequence for Searching





Part V: Version Control Demo



All Commits

Fixed filtering to work properly, shows top 25 songs that are closest to filter

Johnathan-Lavoie committed last week

Added SongFilter.java back and reverted Song.java to required version

Johnathan-Lavoie committed last week

Commits on Mar 12, 2026

Merge pull request #23 from Focusucof/devin

Johnathan-Lavoie authored 2 weeks ago

Added saving to and loading from files using json formatting in the test_song.json file, currently it v

Johnathan-Lavoie committed 2 weeks ago

Added the SongFilter class which allows you to create a filter that can score a song based off of ho

Johnathan-Lavoie committed 2 weeks ago

Added some sample code for filtering songs and saving songs

Johnathan-Lavoie committed 2 weeks ago

Switched to JDK 25 for some nice-to-haves

Johnathan-Lavoie committed 2 weeks ago

Search Songs UML Sequence Diagram

Ashwins1971 authored last week

Commits on Mar 13, 2026

S14 AddSong UML sequence Diagram

Ashwins1971 authored 2 weeks ago

Add files via upload

Ashwins1971 authored 2 weeks ago

Commits on Mar 12, 2026

Create diagrams folder

Ashwins1971 authored 2 weeks ago

Commits

master

All usersAll time

Commits on Mar 26, 2026

Merge pull request #40 from Focusucof/devin

Focusucof authored 17 hours ago

Verifiede53fc17

docker build and docker compose files

Focusucof committed 17 hours ago

75f019d

Commits on Mar 21, 2026

Merge pull request #39 from Focusucof/devin

Focusucof authored last week

Verifiede951c95

Merge branch 'master' into devin

Focusucof committed last week

813688d

fixed all the issues with integration

Focusucof committed last week

f3e25e3

migrating stuff

Focusucof committed last week

8a01c50

can now upload images instead of image urls

Focusucof committed last week

139c583

homepage now shows real data not mock data, searching by title in testing

Focusucof committed last week

a3c3651

Merge pull request #32 from Focusucof/marji

maryammarji authored last week

Verifiedd2ee949

Resolved merge conflict (README)

maryammarji committed last week

553a244

Delete Lab 07 Presentation .pptx

maryammarji committed last week

f1a9c25

Merge pull request #40 from Focusucof/devin

Focusucof authored 16 hours ago

docker build and docker compose files

Focusucof committed 16 hours ago

Commits on Mar 21, 2026

Merge pull request #39 from Focusucof/devin

Focusucof authored last week

Merge branch 'master' into devin

Focusucof committed last week

fixed all the issues with integration

Focusucof committed last week

migrating stuff

Focusucof committed last week

Commits on Mar 20, 2026

Merge pull request #38 from Focusucof/Johnathan

Focusucof authored last week

Merge pull request #32 from Focusucof/marji

maryammarji authored last week

Resolved merge conflict (README)

maryammarji committed last week

Delete Lab 07 Presentation .pptx

maryammarji committed last week

Add files via upload

maryammarji committed last week

Possible Backend Logic for Genre Filtering

maryammarji committed last week

Updated README File for backend

maryammarji committed last week

Merge remote-tracking branch 'origin/marji' into marji

maryammarji committed last week

Possible Backend Logic for Genre Filtering

maryammarji committed last week

Delete Lab 07 Presentation .pptx

maryammarji authored last week

Add files via upload

maryammarji authored last week



Part VI: Test Driven Development



IN-PROGRESS FEATURE: S7 Search by Album

Step 1: Writing a Failing Test (RED) ●

```
@Test
public void testSearchByAlbum() {
    assertEquals("Expected Song", searchByAlbum("AlbumName"));
}
```

Step 2: Adding More Test Suites ●

No matches found

```
@Test
public void testSearchByAlbum_noMatches() {
    MusicLibrary library = new MusicLibrary();
    library.addSong(new Song("Song1", "Artist1", "Album1"));
    List<Song> results = library.searchByAlbum("Nonexistent");
    assertTrue(results.isEmpty());
}
```

Case-insensitive Search

```
@Test
public void testSearchByAlbum_caseInsensitive() {
    MusicLibrary library = new MusicLibrary();
    library.addSong(new Song("Song1", "Artist1", "After Hours"));
    List<Song> results = library.searchByAlbum("after hours");
    assertEquals(1, results.size());
}
```



Step 3: Implementing Simplest Code to Pass Test

```
public List<Song> searchByAlbum(String albumName) {  
    List<Song> results = new ArrayList<>();  
  
    for (Song song : songs) {  
        if (song.getAlbum().toLowerCase().contains(albumName.toLowerCase())) {  
            results.add(song);  
        }  
    }  
  
    return results;  
}
```

● Step 4: Refactor (REFACTOR phase)

Cleaning the code without over complication

- We extract any helper methods if needed
- Improve performance later
- Keep tests passing at all times



● Step 4: Refactor (REFACTOR phase)

Cleaning the code without over complication

- We extract any helper methods if needed
 - Improve performance later
 - Keep tests passing at all times



Part VI: JUnit Testing Demo



Lab Check-List

- ~~PART IV: Create build files and scripts for your project.~~
 - ~~— Write a script (windows or Unix) that allows your project to be built, run, deployed on the server, or some other aspect of setup.~~
 - ~~— Include documentation on how to run your build scripts.~~
 - ~~— If we were a new developer, how would we compile and test/run your program?~~
 - ~~For a feature you have yet to develop, write a Test for it first as described in the Test Driven Development lecture. Demonstrate it to the customer. Run the test showing that it is "Red"~~
 - ~~Then write a test suite that covers your new feature.~~
 - Illustrate your updated task board and your burn down chart.
 - Show us your burn down.
 - What is your updated velocity?
 - ~~— AGILE BOARD: Show us (Screenshots are fine) your project to update.~~
-

- PART VI: DONE IN PERSON + SCREENSHOTS Present your unit tests deliverables
 - What interesting bugs did you find?
 - What other observations do you have? Eg., was testing difficult, useful, et cetera?
- DONE IN PERSON Demonstrate the main trunk of work to the customer and any branches.
- ~~PART V: DONE IN PERSON Demonstrate your fully functional version control system.~~
 - ~~— Every member of the group must have checked in something.~~
- DONE IN PERSON: EVERYONE COMMIT WHAT THEY NEED TO BEFORE THE LAB